

420-SN1-RE Final Exam
Question Booklet
John Abbott College
Winter 2026

Full name: _____

Student ID: _____

Teacher name: _____

INSTRUCTIONS: Read the all of the following instructions carefully.

- Keep this booklet closed until instructed by the exam officiant.
- Write your Full Name and Student ID in the space provided on both the Question Booklet and Answer Booklet.
- Write all answers for each question in the Answer Booklet.
- You may use the Question Booklet for scrap work.
- The last page of the Question Booklet is a Reference Sheet. You can tear the reference sheet away from the booklet.

PERMITTED MATERIALS: This examination is CLOSED BOOK. Only the following resources are permitted:

- Language dictionaries
- Non-programmable calculators.

NO OTHER RESOURCES PERMITTED: including course notes, computers, electronics, etc.

Question 1: Mystery Function (2 marks)

Consider the code below, and then answer the multiple choice questions:

```
def mystery(words):
    result = []

    for w in words:
        if len(w) > 3:
            result.append(w)

    return result

text = [
    "It", "was", "a", "bright", "cold", "day", "in",
    "April", "and", "the", "clocks", "were", "striking", "thirteen"
]
result = mystery(text)
print(result)
```

Q1.A: (1 mark)

What is the return type of the function *mystery* above?

A. `None`

B. `str`

C. `int`

D. `list`

Q1.B: (1 mark)

What is the output of the program above?

A. `["was","bright","cold","day","April","and","the","clocks","were","striking","thirteen"]`

B. `["thirteen","striking","were","clocks","April","cold","bright"]`

C. `"bright cold April clocks were striking thirteen"`

D. `["bright","cold","April","clocks","were","striking","thirteen"]`

Question 2: Graphing (3 marks)

Match each of these programs to the correct result:

Program 1

```
import matplotlib.pyplot as plt

def draw_graph():
    values_1 = [1.0, 2.0, 3.0, 4.0, 5.0]
    values_2 = [0.1, 0.2, 0.3, 0.5, 0.8]
    plt.plot(values_1, values_2)
    plt.title("W2026")
    plt.show()
```

Program 2

```
import matplotlib.pyplot as plt

def draw_graph():
    values_1 = [1.0, 2.0, 3.0, 4.0, 5.0]
    values_2 = [0.1, 0.2, 0.3, 0.5, 0.8]
    plt.title("W2026")
    plt.plot(values_2, values_1)
    plt.show()
```

Program 3

```
import matplotlib.pyplot as plt

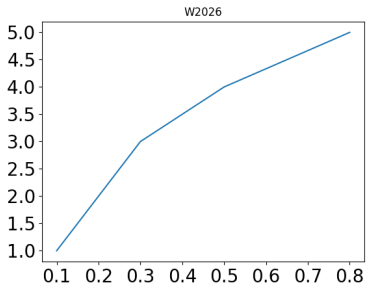
def draw_graph():
    values_1 = [1.0, 2.0, 3.0, 4.0, 5.0]
    values_2 = [0.1, 0.2, 0.3, 0.5, 0.8]
    plt.scatter(values_1, values_2)
    plt.title("W2026")
    plt.show()
```

Program 4

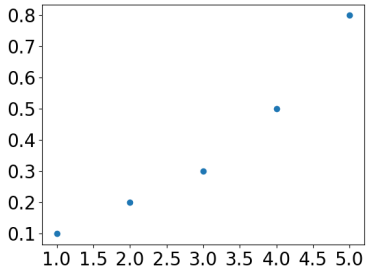
```
import matplotlib.pyplot as plt

def draw_graph():
    values_1 = [1.0, 2.0, 3.0, 4.0, 5.0]
    values_2 = [0.1, 0.2, 0.3, 0.5, 0.8]
    plt.scatter(values_1, values_2)
    plt.show()
    plt.title("W2026")
```

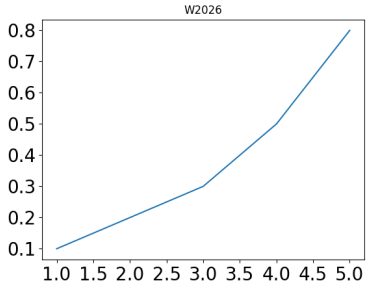
Result A



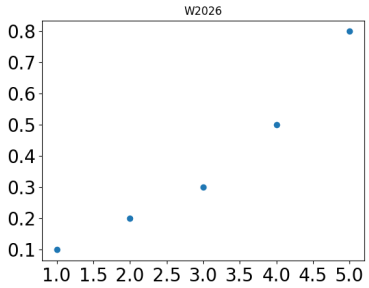
Result B



Result C



Result D



Question 3: Every `x` has `y` (6 marks)

You can either use **lists** or **numpy arrays** for this question. **There is no need to print the created lists.**

Q3.A: (3 marks)

Write python code to create a list of x values of length $N = 1001$ going from `0` to `100` with increments of `0.1`

Q3.B: (3 marks)

Write python code to create a list of y values applying the formula below to all x values produced in part (**A**).

$$f(x) = \frac{3x^3 + x}{x^2 + 2} \tag{1}$$

Question 4: Disk Volumes (5 marks)

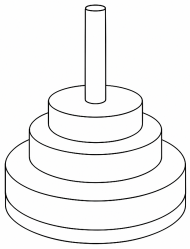
You are given a stack of disks of different radii. Write a function `disk_volume()` that has two parameters:

- `thickness`: a number representing the thickness all disks (every disk has the same thickness)
- `radii`: a list of radius values

Given these parameters, `disk_volume()` should return the total volume of all the disks.
For example:

```
total_volume = disk_volume(1.5, [1, 2, 3])

# prints 14
print(total_volume)
```



A sample image of the shape involved.

Note that there may be more or less disks than shown in the picture.
The formula for the volume of a disk is:

$$v = r^2 \pi t \tag{2}$$

where `v` is the volume, `r` is the disk radius, and `t` is the disk thickness.

Question 5: Types (3 marks)

Write down the **type** AND **value** of the data stored in the `result` variable in each of the following lines of code.

```
result1 = 5 + .4 + 4 + 0.0

result2 = (2+2) >= (2*2)

result3 = 60 / 20

result4 = "20" + "26"

result5 = (3**2 > 3*2) and (2**2 > 2*2)

result6 = 2 * 2**2 * 2
```

Question 6: Code Tracing (6 marks)

For each of the following blocks of code, write down what would be printed when the code is executed.

Q6.A: (2 marks)

```
data_1 = [2,4,1,3]
data_2 = [4,3,2,3]
for i in range(4):
  if data_1[i] == data_2[i]:
    print("beep")
  elif data_1[i] >= data_2[i]:
    print("boop")
  elif data_1[i] <= data_2[i]:
    print("whoop")
  else:
    print("Uh oh")
```

Q6.B: (2 marks)

```
def foo(a,b,c):
  if a and ((not b) or c):
    print("a")
  if b and c:
    print("b")
  elif b and not (a or c):
    print("c")
  else:
    print("d")

foo(True,False,False)
foo(False,True, True)
```

Q6.C: (2 marks)

```
nums = [ -4,3,12,2,5]
total = 0
for i in range(3):
  nums[i] = nums[3]+4
  total = total + nums[i]
print(nums)
print(total)
```

Question 7: Fix mistakes (6 marks)

Each code snippet below has *one or more errors* that prevent it from working as intended.

In the **Answer booklet**, for each part, **circle** the line(s) to fix and **re-write only the affected lines**.

Q7.A: (1 mark)

```
def foo(a,b):
    if a < b :
        return b - a
    else:
        return a - b

result = foo("7","9")
print(result)
```

Intended output:

```
2
```

Q7.B: (1 mark)

```
my_list = [2,4,4,6,7,7,2,3]
for i in range(my_list):
    print(i)
```

Intended output:

```
2
4
4
6
7
7
2
3
```

Q7.C: (2 marks)

```
def find_dosage(weight):
    dosage = 0
    if 0 < weight < 60:
        dosage = 0.1
    if weight < 90:
        dosage = 0.5
    else:
        dosage = 1

    return dosage
```

Intended output:

```
print(find_dosage(30))  #displays 0.1
print(find_dosage(70))  #displays 0.5
print(find_dosage(95))  #displays 1
```

Q7.D: (2 marks)

```
def make_repeated_list(base, n):
    result = []
    for i in range(n):
        result.append(base)

    print(result)
```

Intended output:

```
ones = make_repeated_list(1, 3)
print(ones)    # [1,1,1]
```

Question 8: Helping a Friend (4 marks)

A friend of yours is new to Python and is struggling with their code. They asked you for help fixing their mistakes.

Q8.A: (2 marks)

They tried to write a function that returns the total distance travelled by an object given its initial velocity, acceleration, and time:

```
function calc_distance(a, v_i, t);
    d = v_i x T + at^2/2
    print(calc_distance)
```

Rewrite the code above to produce a working function. You should keep the same function name `calc_distance` and parameters (acceleration `a`, initial velocity `v_i`, time `t`).

The function should return the distance `d` travelled according to the equation of motion: $d = v_i t + (at^2)/2$

Q8.B: (2 marks)

Later, they are trying to use the function to see if a rocket would travel at least 1000 meters. They would like to print a message saying if the rocket will be successful or not. Assume a, v_i, and t are all measured in meters for distance and seconds for time.

```
#assume calc_distance is working and available
t = 120 #seconds of flight time
a = 10 #meters per second squared
v_i = 0 #meters per second

calc_distance(t, a, v_i)

nasa_min = "1000"

if d = nasa_min:
    print("Yay! :)")

print("Boo! :(")
```

Rewrite the code above so that it uses `calc_distance` correctly and produces the desired result.

Question 9: Lists Ahoy (3 marks)

Study the following definition then answer the multiple choice questions:

```
listOfList = [["Huey","Duck",2],["Louie","Quack",3],["Dewey","Duck",4]]
```

Q9.A: (1 mark)

How do you find out home many entries are in listOfList?

A. `listOfList.length()`

B. `len(listOfList[])`

C. `len(listOfList)`

D. `listOfList["Louie"][0]`

Q9.B: (1 mark)

Select two ways that you could use to select the number 3 from within listOfList:

A. `listOfList[1][2]`

B. `listOfList[-2][-2]`

C. `listOfList[-2][-1]`

D. `length(listOfList.get(4))`

E. `length(listOfList.return(4))`

Q9.C: (1 mark)

Would Python allow this?

```
listOfList.append([1,2,3,4])
```

A. `Yes`

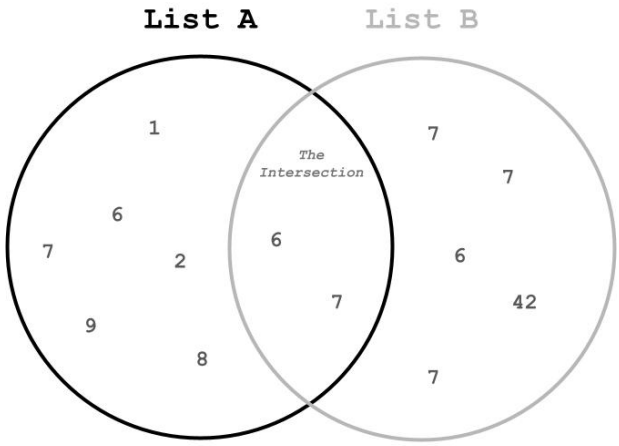
B. `No`

Question 10: Intersection (8 marks)

The **intersection** of two lists is the list of elements that appear in both, with no duplicates, as shown in the figure below.

Write a function called `intersection` that:

- Accepts two lists, `list_a` and `list_b`, as parameters
- Returns a list containing the intersection of the two lists, *without duplicates*



A graphical depiction of the intersection between two lists.

Example: Consider the diagram above. The two lists could be represented in python as:

```
list_a = [1, 6, 2, 7, 2, 6, 7, 8, 9]
list_b = [7, 7, 6, 7, 6, 42, 7]
```

If `intersection()` were called on these two lists, `[6, 7]` would be returned.

HINT: Use `in` to verify if a value is in a list, for example:

```
if 5 in [1,2,5,7,8]:
    print("It's here!")
else:
    print("It's not here!")

# Result: "It's here!" is printed
```

Question 11: Parse Veterinary Data (8 marks)

You were recently hired as an intern at a Vet clinic to help make sense of data stored in a CSV file. Your task is to read pet information from the `"dataset_pets.csv"`.

Here are the **first few lines** of this file (it contains many more lines)

```
Pet Name,Animal Type,Age (in years),Weight,Vaccines
Meelo,Cat,0.3,1.9,Feline Distemper (FVRCP)
Thom,Dog,1.2,3.4,Canine parvovirus
Gerald,Rat,0.2,0.9,Rabies
...
```

Q11.A: (3 marks)

For this question, starter code is provided in the Answer Booklet. Your task is to complete the starter code.

Read each of the 5 columns of the CSV file into the lists variables provided in the answer booklet.

I.e., all of the pet names should be put into the `names` list, all of the animal types should be put into the `types` list, etc.

HINT: Ensure that `weights` and `ages` are lists of numbers.

Q11.B: (5 marks)

Write code to determine and **print**:

- the **number of dogs** (animal type is `"Dog"`)
- the **average age** of dogs
- find a list of vaccines given to Dogs at this clinic. The list of vaccines must be made up of **unique values**.

Question 12: Genetic Sequence (6 marks)

In genetics, the *TATA* sequence is a promoter region that initiates DNA transcription (copied to start making a new protein).

The following list contains the exact DNA sequence of the TATA region (assume that this will not change):

```
TATA = ['T', 'A', 'T', 'A', 'A', 'A']
```

A full sequence of genetic material is a much longer list.

```
full_dna = ['A','T','G','C','G','T','A','T','A','A','A','G','A','C','C','T','A','T', ...]
```

Write a function `has_tata()` which takes the full DNA sequence as input parameter and returns `True` if the TATA sequence is found, otherwise returns `False`.

Here's how this function can be used:

```
full_dna = ['A','T','G','C','G','T','A','T','A','A','A','G','A','C','C','T','A','T']
result = has_tata(full_dna)
print(result)  # output: True
```

Reference Sheet

Math

```
# import math library to use math functions
import math

result = math.ceil(6.1)  # result == 7
result = math.floor(6.1) # result == 6
result = abs( -7)         # result == 7
result = math.sqrt(9)    # result == 3.0

# trigonometry
theta = math.pi           # theta == pi
result = math.sin(theta)  # result == 0.0
```

Arithmetic Operators

```
# add, subtract, multply, divide, exponentiate
result = 20 + 25   # result == 45
result = 20 - 25   # result == -5
result = 20 / 25   # result == 0.8
result = 4 / 2     # result == 2.0
result = 20 * 25   # result == 500
result = 2 ** 4    # result == 16
result = "a" + "b" # result == "ab"

# integer division
result = 20 // 25   # result == 0
result = 4 // 2     # result == 2
result = 4.0 // 2   # result == 2.0
result = 4 // 2.0   # result == 2.0

# modulus - - returns the remainder after division
result = 20 % 2   # result == 0
result = 25 % 2   # result == 1
result = 25 % 20  # result == 5
```

Comparisons and Logical Operators

```
# Equal to `==`, not equal to `!=`
result = 20 == 25   # result == False
result = 20 != 25   # result == True

# Less than `<`, greater than `>`,
result = 20 > 25   # result == False
result = 20 < 25   # result == True
result = 20 < 20   # result == False

# less than or equal to `<=`
# greater than or equal to `>=`
result = 20 <= 20   # result == True

# combining booleans using 'and'
result = 20 == 25 and 20 > 25  # result == False
result = 20 != 25 and 20 < 25  # result == True

# combining booleans using 'or'
result = 20 == 25 or 20 > 25  # result == False
result = 20 != 25 or 20 > 25  # result == True

# something is `in` a list, something is `not in` a list
result = 25 in [20, 25]       # result == True
result = 25 in [20, 26]       # result == False
result = 25 not in [20, 26]   # result == True
```

Type Conversions

```
result = int(4.2)           # result == 4
result = float("4.2")       # result == 4.2
result = str(4.2)           # result == "4.2"
```

Numpy

```
import numpy as np
# Create an array from a list
xs_arr = np.array([1,2,3])
# Create an array from another array
ys_arr = 3*xs_arr + 2  # result: np.array([5,8,11])
# Add arrays
z_arr = x_arr + y_arr  # result: np.array([6,10,14])
```

Looping

```
# i goes from 0 .. 9
for i in range(10):
# i goes from 3 .. 9
for i in range(3, 10):
# i gets the value 3, then 5, then 7, then 9
for i in range(3, 10, 2):
# fruit gets the value "apple", then "banana", then "orange"
for fruit in ["apple", "banana", "orange"]:

# fruit gets the value "apple", then "banana", then "orange"
# index gets the value 0, then 1, then 2
for index, fruit in enumerate(["apple", "banana", "orange"]):

# zip allows you to go through multiple lists simultaneously
for a, b in zip(a_list, b_list):
for a, b, c in zip(a_list, b_list, c_list):

# break to exit a for loop early
# continue restarts the loop with the following element
for some_var in some_list:
  if some_condition:
    break
  if other_condition:
    continue #restarts the loop with the next element
```

Functions

```
# An example function definition:
def foo(a, b):     # function 'foo' takes two parameters 'a' and 'b'
    return a + b   # returns the result of adding 'a' and 'b'

# Using the example function:
result = foo(1, 2)      # result gets the value 3
result = foo("1", "2")  # result gets the value "12"
```

Lists

```
# Create lists:
my_list = []                              # Empty
my_list = ["apple", "banana", "orange"]  # Initial values

# Access a specific element in a list using an index:
result = my_list[0]         # result == "apple"
my_list[0] = "pear"         # my_list == ["pear", "banana", "orange"]

# Use negative index to get value in list in reverse order:
result = my_list[ -1]        # result == "orange"

# Get the length of a list:
result = len(my_list)       # result == 3

# Add something to the end of a list:
my_list.append("lime")

# Get the maximum / minimum / sum of a list's items:
number_list = [1, 2, 3]
result = max(number_list)   # result == 3
result = min(number_list)   # result == 1
result = sum(number_list)   # result == 6

# Remove something from the end of a list:
result = number_list.pop()  # result == 3, number_list == [1, 2]

# And then remove something from a given index of a list:
result = number_list.pop(0) # result == 1, number_list == [2]

# Sort/reverse a list:
number_list = [3, 2, 1]
number_list.sort()       # number_list == [1, 2, 3]
number_list.reverse()    # number_list == [3, 2, 1]
```

CSV File

```
# A CSV file has the following structure on each line
# column1, column2
import csv
with open("example.csv", 'r') as file:
  reader = csv.reader(file)
  for line in reader:
    print(line)        # each line is a list: ["column1", "column2"]
```